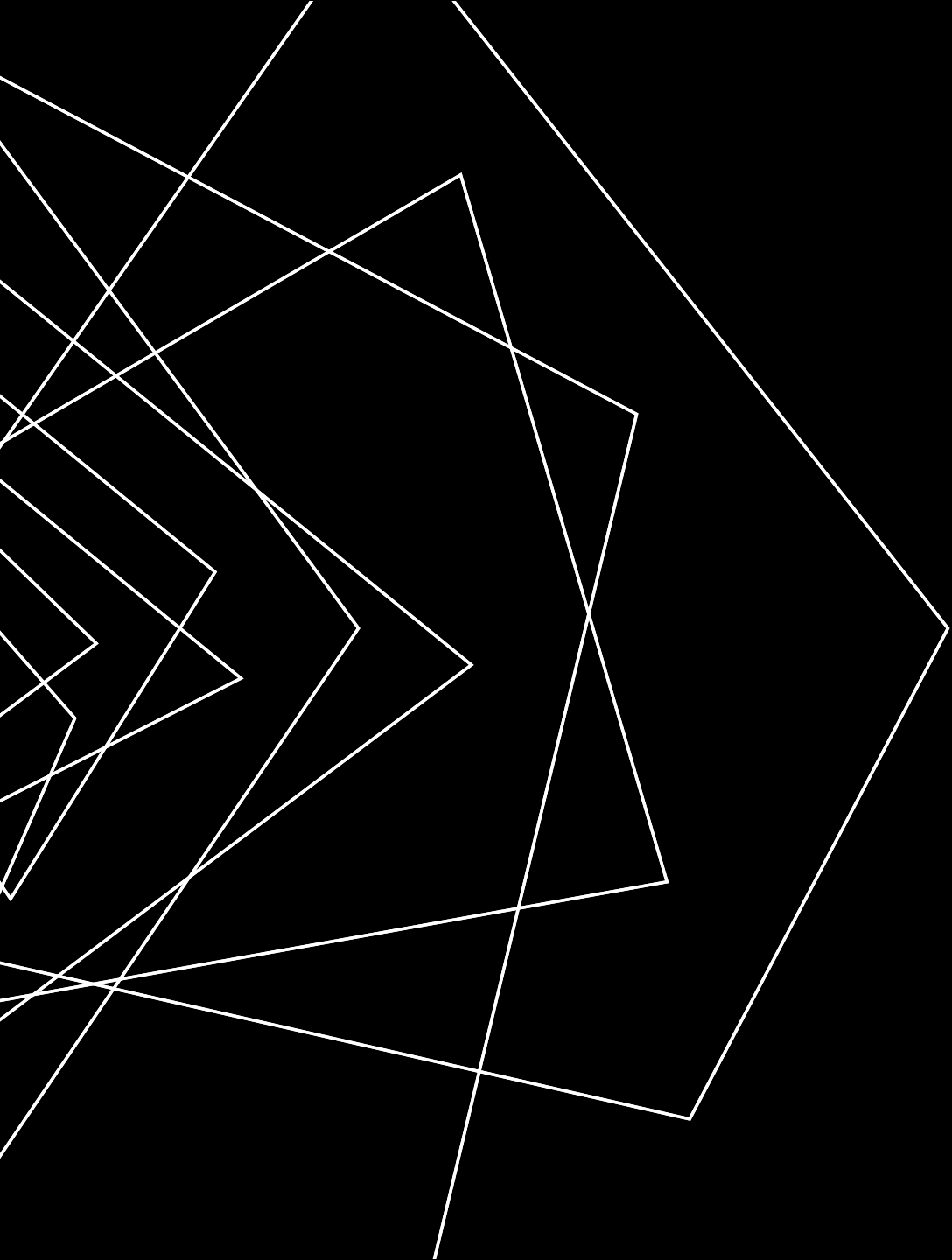




EAST TEXAS A&M
— UNIVERSITY —

CSCI 380.61E
WEB PROGRAMMING
AND INTERFACE DESIGN



ARCHITECTURE, MONGOOSE, CRUD

Web Programming and Interface Design
- MongoDB

LEARNING OBJECTIVES

- Understand MongoDB architecture.
- Use Mongoose.
- CRUD.
- Integrating CRUD Operations with Hapi.js.

MONGODB

- MongoDB is a NoSQL database that stores data in flexible, JSON-like documents.
- Key Features:
 - Schema-less
 - High availability and scalability
 - Rich querying capabilities

MONGODB

- Use Cases:
 - Real-time analytics
 - Content management systems
 - Internet of Things (IoT)

MONGODB ARCHITECTURE

- Components:
 - Database
 - Collection
 - Document

MONGODB ARCHITECTURE

- Data Model:
 - Documents and collections
- ```
{
 "_id": "unique_id",
 "name": "John Doe",
 "email": "john@example.com",
 "age": 30
}
```

# MONGOOSE

- Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js.
- Benefits:
  - Schema validation
  - Middleware support
  - Easy to use APIs for CRUD operations
- Installation: `npm install mongoose`

# CONNECTING TO MONGODB WITH MONGOOSE

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost:27017/mydatabase', {
 useNewUrlParser: true
}).then(() => {
 console.log('MongoDB connected');
}).catch(err => {
 console.error(err);
});
```

# DEFINING A MONGOOSE SCHEMA

```
const { Schema, model } = require('mongoose');

const userSchema = new Schema({
 name: { type: String, required: true },
 email: { type: String, required: true, unique: true },
 age: { type: Number, min: 0 }
});

const User = model('User', userSchema);
```

# CRUD OPERATIONS OVERVIEW

- CRUD
  - Create
  - Read
  - Update
  - Delete
- Using Mongoose for CRUD Operations

# CREATE OPERATION

```
const createUser = async (userData) => {
 const user = new User(userData);
 return await user.save();
};
```

# READ OPERATION

- Find All Users

```
const getAllUsers = async () => {
 return await User.find();
};
```

# READ OPERATION

- Find a User by ID

```
const getUserById = async (id) => {
 return await User.findById(id);
};
```

# READ OPERATION

- Find Users with Query Filters

```
const getUsersByAge = async (age) => {
 return await User.find({ age: { $gte: age } }); // Users older than
 or equal to specified age
};
```

# READ OPERATION

- Find One User

```
const getOneUser = async (filter) => {
 return await User.findOne(filter); // Finds the first user that
 matches the filter
};
```

# READ OPERATION

- Using Logical Operators

```
const getUsersWithConditions = async () => {
 return await User.find({
 $or: [
 { age: { $lt: 18 } }, // Users younger than 18
 { name: /Doe$/ } // Users whose names end with "Doe"
]
 });
};
```

# READ OPERATION

- Paginated Results

```
const getPaginatedUsers = async (page, limit) => {
 return await User.find()
 .skip((page - 1) * limit)
 .limit(limit);
};
```

# UPDATE OPERATION

- Update User by ID

```
const updateUserById = async (id, userData) => {
 return await User.findByIdAndUpdate(id, userData, { new: true,
runValidators: true });
};
```

# UPDATE OPERATION

- Update Multiple User

```
const updateUserByAge = async (age, updateData) => {
 return await User.updateMany({ age: { $lt: age } }, updateData);
 // Update all users younger than specified age
};
```

# UPDATE OPERATION

- Update User's Specific Field

```
const updateUserEmail = async (id, newEmail) => {
 return await User.findByIdAndUpdate(id, { email: newEmail }, {
 new: true });
};
```

# UPDATE OPERATION

- Using findOneAndUpdate

```
const findOneAndUpdateUser = async (filter, updateData) => {
 return await User.findOneAndUpdate(filter, updateData, { new:
 true, runValidators: true });
};
```

# UPDATE OPERATION

- Using findOneAndUpdate

```
const findOneAndUpdateUser = async (filter, updateData) => {
 return await User.findOneAndUpdate(filter, updateData, { new:
 true, runValidators: true });
};
```

# UPDATE OPERATION

- Conditional Update

```
const conditionalUpdate = async (id, userData) => {
 const user = await User.findById(id);
 if (user.age < 18) {
 return await User.findByIdAndUpdate(id, { adult: false }, {
new: true });
 }
 return await User.findByIdAndUpdate(id, { adult: true }, { new:
true });
};
```

# DELETE OPERATION

```
const deleteUser = async (id) => {
 return await User.findByIdAndDelete(id);
};
```

# INTEGRATING CRUD OPERATIONS WITH HAPI.JS

```
server.route([
 {
 method: 'POST',
 path: '/users',
 handler: async (request, h) => {
 const user = await createUser(request.payload);
 return h.response(user).code(201);
 }
 }
]);
```

# ERROR HANDLING

- Use try-catch blocks
- Return appropriate HTTP status codes

# ERROR HANDLING

```
server.route({
 method: 'GET',
 path: '/users/{id}',
 handler: async (request, h) => {
 try {
 const user = await User.findById(request.params.id);
 if (!user) {
 return h.response({ message: 'User not found'
}).code(404);
 }
 }
 }
});
```

# ERROR HANDLING

```
}
 return user;
 } catch (error) {
 return h.response({ message: 'Error retrieving user'
}).code(500);
 }
}
});
```

# VALIDATION WITH MONGOOSE

- Using Schema Validation:
  - Built-in validation features

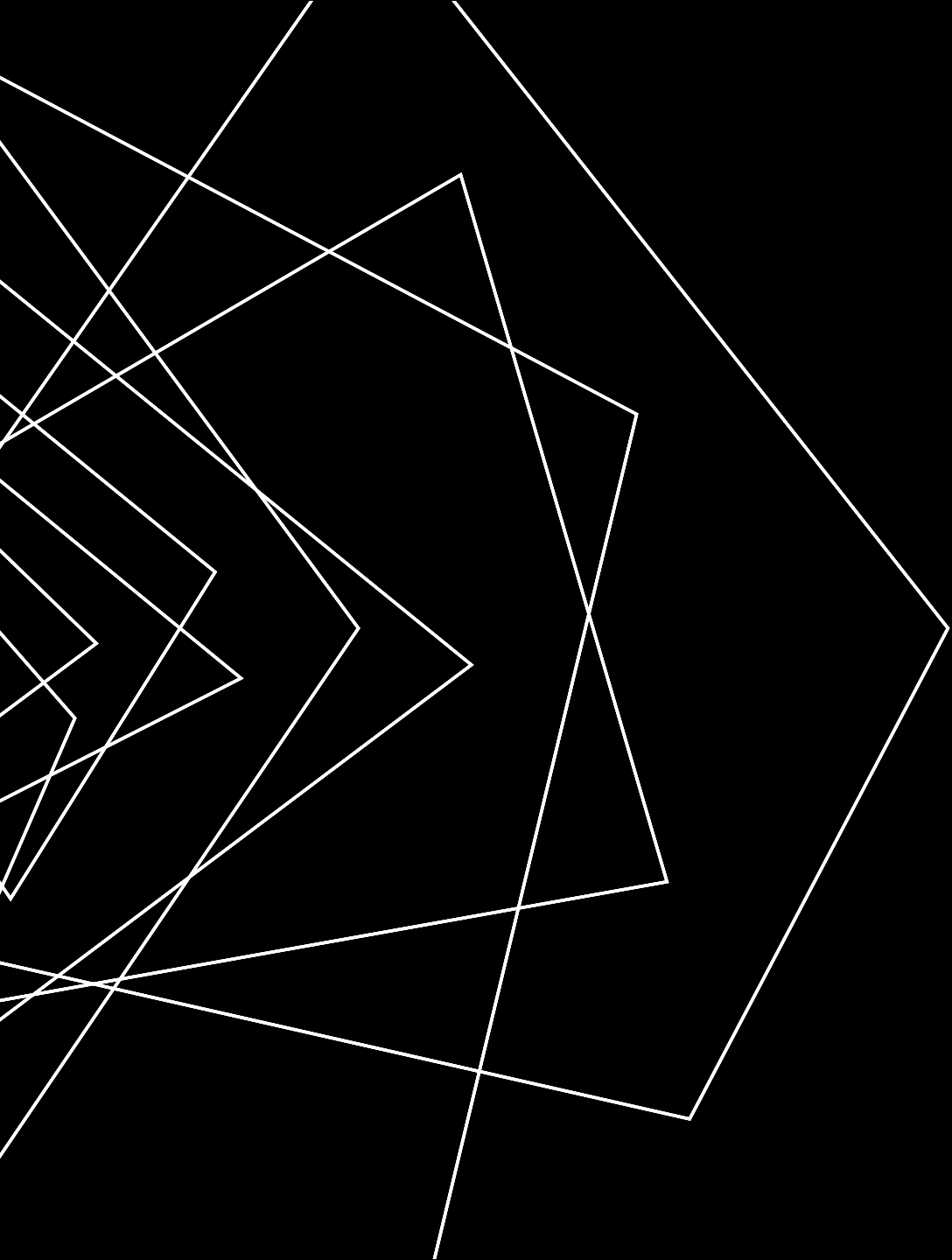
```
const userSchema = new Schema({
 name: { type: String, required: true },
 email: { type: String, required: true, unique: true, match:
 /\.+\@.+\.+\/ },
 age: { type: Number, min: 0 }
});
```

# BEST PRACTICES

- Connection Management:
  - Properly manage MongoDB connections
- Error Handling:
  - Consistent error handling strategies
- Validation:
  - Always validate input data

# LAB

- Create MongoDB databse.
- CRUD API for a resource (Books, Users).



# THANK YOU

Yau Ka Cheung  
Yauka.Cheung@etamu.edu  
ACB1 Room 335